> https://codeforces.com/contest/2060

A. Fibonacciness

> https://codeforces.com/contest/2060/problem/A

time limit per test: 1 second
memory limit per test: 256 megabytes

There is an array of 5 integers. Initially, you only know a_1, a_2, a_4, a_5. You may set a_3 to any positive integer, negative integer, or zero. The Fibonacciness of the array is the number of integers i ($1 \le i \le 3$) such that $a_{i+2} = a_i + a_{i+1}$. Find the maximum Fibonacciness over all integer values of a_3.

有一个由 5 个整数组成的数组。最初，你只知道 a_1, a_2, a_4, a_5 。你可以将 a_3 设置为任何正整数、负整数或零。数组的 Fibonacciness 是满足 $a_{i+2} = a_i + a_{i+1}$ ($1 \le i \le 3$) 的整数i的个数。求 a_3 的所有整数值的最大 Fibonacciness。

Input

The first line contains an integer t ($1 \le t \le 500$) — the number of test cases.

The only line of each test case contains four integers a_1, a_2, a_4, a_5 ($1 \le a_i \le 100$).

第一行包含一个整数 t ($1 \le t \le 500$) - 测试用例的数量。

每个测试用例的唯一一行包含四个整数 a_1, a_2, a_4, a_5 ($1 \le a_i \le 100$)。

Output

For each test case, output the maximum Fibonacciness on a new line.

对于每个测试用例，在新行中输出最大 Fibonacciness 值。

题解

为了最大化 Fibonacciness ，a_3 取得肯定是 $a_1 + a_2$、$a_4 - a_2$、$a_5 - a_4$中的一个。分别计算这些取值下的 Fibonacciness 即可。

代码如下

```
t = int(input())
for _ in range(t):
    a1, a2, a4, a5 = map(int, input().split())
    a3 = a1 + a2
    ans1 = 1
    if a2 + a3 == a4:
        ans1 += 1
    if a3 + a4 == a5:
        ans1 +=1
    a3 = a4 - a2
    ans2 = 1
    if a1 + a2 == a3:
        ans2 += 1
    if a3 + a4 == a5:
        ans2 +=1
    a3 = a5 - a4
```

```
    ans3 = 1
    if a1 + a2 == a3:
        ans3 += 1
    if a2 + a3 == a4:
        ans3 += 1
    print(max(ans1, ans2, ans3))
```

B. Farmer John's Card Game

> https://codeforces.com/contest/2060/problem/B

time limit per test: 2 second
memory limit per test: 256 megabytes

Farmer John's n cows are playing a card game! Farmer John has a deck of $n \cdot m$ cards numbered from 0 to $n \cdot m - 1$. He distributes m cards to each of his n cows.

Farmer John wants the game to be fair, so each cow should only be able to play 1 card per round. He decides to determine a turn order, determined by a permutation* p of length n, such that the p_i'th cow will be the i'th cow to place a card on top of the center pile in a round.

In other words, the following events happen in order in each round:

- The p_1'th cow places any card from their deck on top of the center pile.
- The p_2'th cow places any card from their deck on top of the center pile.
- ...
- The p_n'th cow places any card from their deck on top of the center pile.

There is a catch. Initially, the center pile contains a card numbered -1. In order to place a card, the number of the card must be greater than the number of the card on top of the center pile. Then, the newly placed card becomes the top card of the center pile. If a cow cannot place any card in their deck, the game is considered to be lost.

Farmer John wonders: does there exist p such that it is possible for all of his cows to empty their deck after playing all m rounds of the game? If so, output any valid p. Otherwise, output -1.

*A permutation of length n contains each integer from 1 to n exactly once

农场主约翰的 n 头牛正在玩纸牌游戏！农场主约翰有一副从 0 到 $n \cdot m - 1$ 编号为 $n \cdot m$ 的扑克牌。他把 m 张扑克牌分给了他的每头 n 奶牛。

农场主约翰希望游戏公平，因此每头牛每轮只能出 1 张牌。他决定确定一个回合顺序，由长度为 p 的排列 * 决定。长度为 n 的排列组合 p 决定，这样第 p_i 头奶牛将是第 i 头在一轮中把牌放在中间牌堆顶上的奶牛。

换句话说，在每一轮中，以下事件会依次发生：

- 第 p_1 头牛将其牌组中的任意一张牌放在中间牌堆的顶端。
- 第 p_2 头牛将其牌组中的任意一张牌放在中间牌堆的顶端。
- ...
- 第 p_n 头牛将其牌组中的任意一张牌放在中间牌堆的顶端。

有一个陷阱。一开始，中心牌堆中有一张编号为 -1 的牌。要放置一张牌，这张牌的编号必须大于中间那张牌的编号。然后，新放的牌就会成为中间牌堆最上面的牌。如果奶牛无法放进自己牌堆中的任何一张牌，游戏就算输了。

农场主约翰想知道：是否存在 p 使得他的所有奶牛在玩了所有 m 轮游戏后都能清空他们的牌组？如果存在，输出任意有效的 p 。否则，输出 -1 。

*长度为 n 的排列恰好包含 1 至 n 中的每个整数一次。

Input

The first line contains an integer t ($1 \le t \le 400$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \le n \cdot m \le 2\,000$) — the number of cows and the number of cards each cow receives.

The following n lines contain m integers each – the cards received by each cow. It is guaranteed all given numbers (across all n lines) are distinct and in the range from 0 to $n \cdot m - 1$, inclusive.

It is guaranteed the sum of $n \cdot m$ over all test cases does not exceed $2\,000$.

第一行包含一个整数 t ($1 \le t \le 400$) - 测试用例的数量。

每个测试用例的第一行包含两个整数 n 和 m ($1 \le n \cdot m \le 2\,000$)--奶牛数量和每头奶牛收到的卡片数量。

下面的 n 行分别包含 m 个整数，即每头牛收到的卡片数。保证所有给定的数字（横跨所有 n 行）都是不同的，且范围在 0 至 $n \cdot m - 1$ 之间。

保证所有测试案例中 $n \cdot m$ 的总和不超过 $2\,000$ 。

Output

For each test case, output the following on a new line:

- If p exists, output n space-separated integers $p_1, p_2, \ldots, p_n$.
- Otherwise, output -1.

对于每个测试用例，在新行中输出以下内容：

- 如果存在 p ，则输出 n 空格分隔的整数 $p_1, p_2, \ldots, p_n$ 。
- 否则，输出 -1 。

题解

因为要满足出的牌按顺序，我们可以现将牛按照获得的牌的最小值排序，并保存当前排列。

接下来检查成出 $n \cdot m$ 张牌的过程中是否都符合条件（为了尽可能满足要求，每次都出牌堆中的最小值）。

代码如下

```
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    cow = [[i+1] + sorted(list(map(int, input().split()))) for i in range(n)]
    cow.sort(key = lambda x:x[1])
    f = 1
    p = [x[0] for x in cow]
    for i in range(n):
        cow[i] = cow[i][1:]
    pre = -1
```

```
    for i in range(n*m):
        idx = i%n
        if min(cow[idx]) > pre:
            pre = min(cow[idx])
            cow[idx].remove(min(cow[idx]))
        else:
            f = 0
            break
    if f:
        print(*p)
    else:
        print(-1)
```

C. Farmer John's Card Game

> https://codeforces.com/contest/2060/problem/C

time limit per test: 2 second
memory limit per test: 256 megabytes

Alice and Bob are playing a game. There are n (n **is even**) integers written on a blackboard, represented by $x_1, x_2, \ldots, x_n$. There is also a given integer k and an integer score that is initially 0. The game lasts for $\frac{n}{2}$ turns, in which the following events happen sequentially:

- Alice selects an integer from the blackboard and erases it. Let's call Alice's chosen integer a.
- Bob selects an integer from the blackboard and erases it. Let's call Bob's chosen integer b.
- If $a + b = k$, add 1 to score.

Alice is playing to minimize the score while Bob is playing to maximize the score. Assuming both players use optimal strategies, what is the score after the game ends?

爱丽丝和鲍勃正在玩一个游戏。黑板上写着 n （n **为偶数**）个整数，用 $x_1, x_2, \ldots, x_n$ 表示。还有一个给定的整数 k 和一个整数分数，分数最初为 0 。游戏持续 $\frac{n}{2}$ 个回合，在此期间会依次发生以下事件：

- 爱丽丝从黑板上选择一个整数并擦除。我们把爱丽丝选择的整数称为 a 。
- 鲍勃从黑板上选中一个整数并擦除。我们把鲍勃选择的整数称为 b 。
- 如果是 $a + b = k$ ，则得分加上 1 。

爱丽丝的目标是最小化分数，而鲍勃的目标是最大化分数。假设双方都使用最优策略，那么游戏结束后的得分是多少？

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \le n \le 2 \cdot 10^5, 1 \le k \le 2 \cdot n$, n is even).

The second line of each test case contains n integers $x_1, x_2, \ldots, x_n$ ($1 \le x_i \le n$) — the integers on the blackboard.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed the sum of $n \cdot m$ over all test cases does not exceed $2\,000$.

第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例的第一行包含两个整数 n 和 k （ $2 \le n \le 2 \cdot 10^5, 1 \le k \le 2 \cdot n$， n 为偶数）。（ $2 \le n \le 2 \cdot 10^5, 1 \le k \le 2 \cdot n$， n 为偶数）。

每个测试用例的第二行包含 n 个整数 $x_1, x_2, \ldots, x_n$ ($1 \le x_i \le n$) - 黑板上的整数。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$ 。

Output

For each test case, output the score if both players play optimally.

对于每个测试案例，如果双方都以最佳状态进行比赛，则输出得分。

题解

因为 n 为偶数，我们总能将牌两两分为一组。Bob 会尽可能最大化比分，因此假如 Alice 选的整数存在 $b = k - a$，Bob 一定会选择 b。所以本题即求有多少对 (a, b) 满足 $a + b = k$ 且每个数只能出现一次。

代码如下

```
t = int(input())
for _ in range(t):
    n, k = map(int, input().split())
    a = list(map(int, input().split()))
    a.sort()
    m = [0 for i in range(2*n+1)]
    t = 0
    for i in range(n):
        m[a[i]] += 1
    for i in range(0, k//2+1):
        if i != k//2 + k%2:
            if m[i]:
                t += min(m[i], m[k-i])
        else:
            if m[i]:
                t += m[i]//2
    print(t)
```

D. Subtract Min Sort

> https://codeforces.com/contest/2060/problem/D

time limit per test: 2 second
memory limit per test: 256 megabytes

You are given a sequence a consisting of n positive integers.

You can perform the following operation any number of times.

- Select an index i ($1 \le i < n$), and subtract $\min(a_i, a_{i+1})$ from both a_i and a_{i+1}.

Determine if it is possible to make the sequence **non-decreasing** by using the operation any number of times.

给你一个由 n 个正整数组成的序列 a 。

您可以执行以下任意次数的运算。

- 选择索引 i ($1 \le i < n$)，然后从 a_i 和 a_{i+1} 中减去 $\min(a_i, a_{i+1})$ 。

判断是否有可能通过任意次数的运算使序列**不递减**。

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \le n \le 2 \cdot 10^5$).

The second line of each test case contains $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

每个测试包含多个测试用例。第一行包含测试用例的数量 t （ $1 \le t \le 10^4$ ）。($1 \le t \le 10^4$).测试用例说明如下。

每个测试用例的第一行都包含一个整数 n ($2 \le n \le 2 \cdot 10^5$)。

每个测试用例的第二行包含 $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$)。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$ 。

Output

If it is possible to make the sequence **non-decreasing**, print "YES" on a new line. Otherwise, print "NO" on a new line.

You can output the answer in any case. For example, the strings "yEs", "yes", and "Yes" will also be recognized as positive responses.

如果可以使序列**不递减**，则在新行上打印 "是"。否则，另起一行打印 "否"。

在任何情况下都可以输出答案。例如，字符串 "yEs"、"yes "和 "Yes "也会被识别为肯定回答。

题解

由于最后要求元素不递减，我们把所有的元素（除最后一个）变为 0 肯定能满足要求。

我们可以最每个位置都操作一次，完成后判断是否不递减。

代码如下

```
t = int(input())
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))
    for i in range(n-1):
        mi = min(a[i], a[i+1])
        a[i] -= mi
        a[i+1] -= mi
    f = 1
    for i in range(1, n):
        if a[i] < a[i-1]:
            f = 0
            break
    if f:
        print('YES')
```

```
    else:
        print('NO')
```

E. Graph Composition

> https://codeforces.com/contest/2060/problem/E

time limit per test: 2 second
memory limit per test: 256 megabytes

You are given two simple undirected graphs F and G with n vertices. F has m_1 edges while G has m_2 edges. You may perform one of the following two types of operations any number of times:

- Select two integers u and v ($1 \le u, v \le n$) such that there is an edge between u and v in F. Then, remove that edge from F.
- Select two integers u and v ($1 \le u, v \le n$) such that there is no edge between u and v in F. Then, add an edge between u and v in F.

Determine the minimum number of operations required such that for all integers u and v ($1 \le u, v \le n$), there is a path from u to v in F **if and only if** there is a path from u to v in G.

给您两个简单的无向图 F 和 G，其中 n 有顶点。 F 有 m_1 条边，而 G 有 m_2 条边。您可以多次执行以下两种操作之一：

- 选择两个整数 u 和 v ($1 \le u, v \le n$)，使得 F 中的 u 和 v 之间有一条边。然后从 F 中删除这条边。
- 选择两个整数 u 和 v （ $1 \le u, v \le n$ ），使得在 F 中的 u 和 v 之间没有边。然后，在 F 中的 u 与 v 之间添加一条边。

求对于所有整数 u 和 v （ $1 \le u, v \le n$ ），在 F 中存在一条从 u 到 v 的路径所需的最小操作数。**当且仅当**在 G 中存在一条从 u 到 v 的路径。

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of independent test cases.

The first line of each test case contains three integers n, m_1, and m_2 ($1 \le n \le 2 \cdot 10^5$, $0 \le m_1, m_2 \le 2 \cdot 10^5$) — the number of vertices, the number of edges in F, and the number of edges in G.

The following m_1 lines each contain two integers u and v ($1 \le u, v \le n$) — there is an edge between u and v in F. It is guaranteed that there are no repeated edges or self loops.

The following m_2 lines each contain two integers u and v ($1 \le u, v \le n$) — there is an edge between u and v in G. It is guaranteed that there are no repeated edges or self loops.

It is guaranteed that the sum of n, the sum of m_1, and the sum of m_2 over all test cases do not exceed $2 \cdot 10^5$.

第一行包含一个整数 t ($1 \le t \le 10^4$) - 独立测试用例的数量。

每个测试用例的第一行包含三个整数 n 、 m_1 和 m_2 ($1 \le n \le 2 \cdot 10^5$, $0 \le m_1, m_2 \le 2 \cdot 10^5$)--顶点数、 F 中的边数和 G 中的边数。

下面的 m_1 行分别包含两个整数 u 和 v （$1 \le u, v \le n$）--在 F 中的 u 和 v 之间有一条边。保证没有重复边或自循环。

下面的 m_2 行分别包含两个整数 u 和 v （$1 \le u, v \le n$）- G 中的 u 和 v 之间存在一条边。保证不存在重复边或自循环。

保证所有测试用例中 n 的总和、 m_1 的总和以及 m_2 的总和不超过 $2 \cdot 10^5$ 。

Output

For each test case, output a single integer denoting the minimum operations required on a new line.

对于每个测试用例，在新行中输出一个整数，表示所需的最小操作数。

题解

题目要求 在 F 中存在一条从 u 到 v 的路径所需的最小操作数。**当且仅当**在 G 中存在一条从 u 到 v 的路径，即F和G的连通分量是相同的（**不考虑边，仅考虑点**）。

对于 F 中每条边，如果两个点不在 G 的同一个连通分量里，我们删边。

接下来，F 满足 G 中没有的路径一定没有，我们只需把 F 中不相连但 G 中是一个的连通分量连接起来即可（即求F, G 连通分量之差）

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef __int128 INT;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
const ll INF = INT_MAX;
const ll MOD = 1000000007;
const ll mod = 998244353;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

ll find(ll p, vector<ll>&parent){
    if (parent[p] != p){
        parent[p] = find(parent[p], parent);
    }
```

```
    return parent[p];
}

void solve(){
    ll n = read(), m1 = read(), m2 = read();
    vector<ll> parentF(n), parentG(n);
    for(ll i=0;i<n;i++){
        parentF[i] = parentG[i] = i;
    }
    vector<PLL> edges;
    ll ans=0;
    for(ll i=0;i<m1;i++){
        edges.push_back({read()-1, read()-1});
    }
    for(ll i=0,u,v;i<m2;i++){
        u = read()-1;
        v = read()-1;
        parentG[find(u, parentG)] = find(v, parentG);
    }
    for(auto [u, v]:edges){
        if (find(u, parentG) == find(v, parentG)){
            parentF[find(u, parentF)] = find(v, parentF);
        }
        else{
            ans++; // u, v不在一个连通分量里,删边
        }
    }
    for(ll i=0;i<n;i++){
        ans += (parentF[i] == i);
        ans -= (parentG[i] == i);
    }
    print(ans);
}

int main(){
    ll t = read();
    while(t--){
        solve();
    }
}
```